

Nombre completo: _____

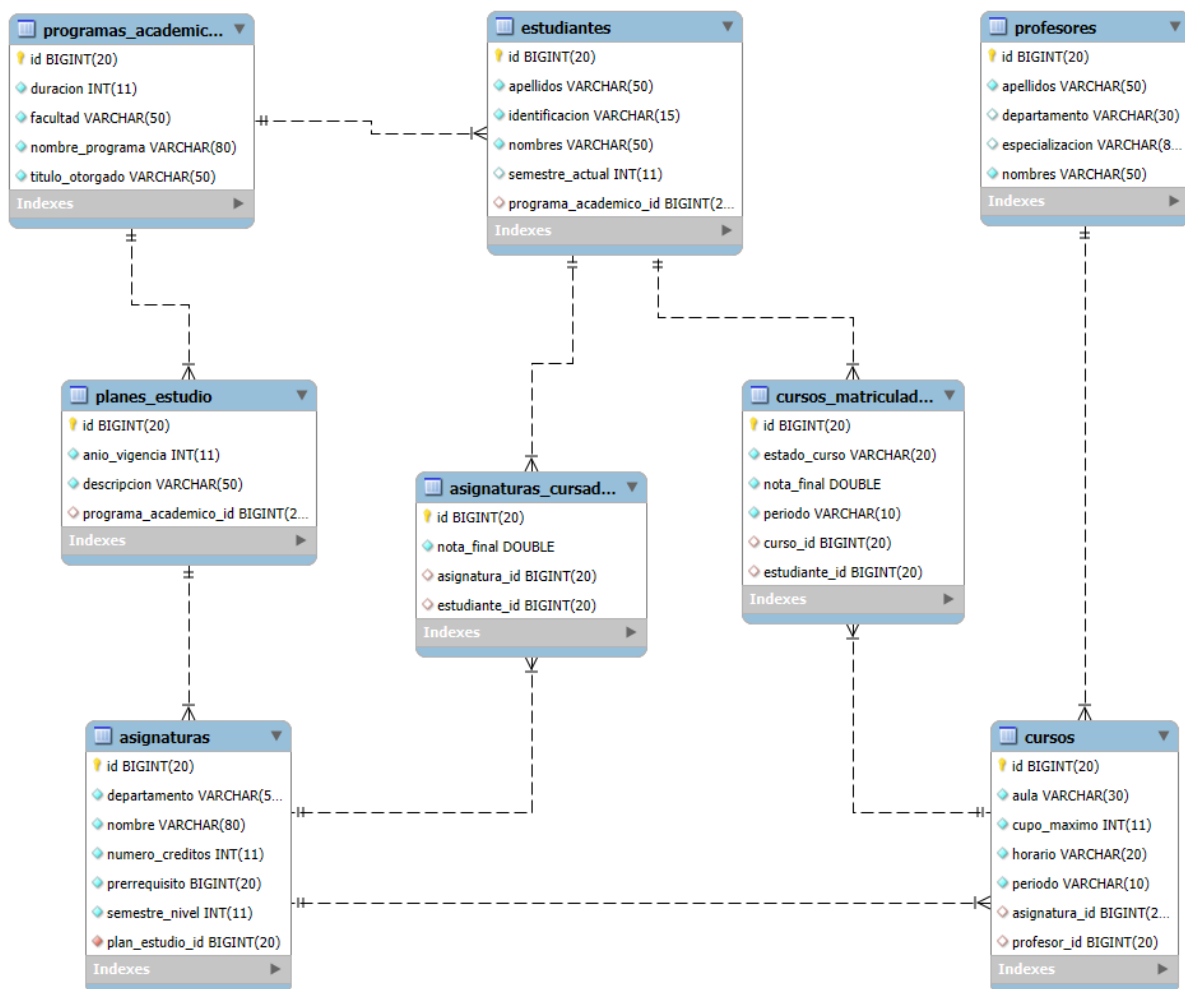
No. Cédula: _____

Fecha: _____

Suficiencia de Lenguaje de programación Avanzado II

Desarrollar una Aplicación Web/Empresarial de tipo **Académico** que permita simular la matrícula de cursos de un estudiante para el siguiente semestre/nivel en un programa académico y almacenar esa matrícula en la tabla "cursos_matriculados" de la siguiente base de datos.

Se debe utilizar como base el siguiente diagrama EER.



Cumplir con los siguientes requisitos:

1. Se debe desarrollar el Front (vista) de un CRUD para **Estudiantes** en donde debe existir una opción que permita matricular los cursos para el siguiente semestre/nivel como se muestra a continuación:

Ejemplo de vista para la tabla de estudiantes:

Listado de estudiantes						
Nuevo estudiante						
Id.	Identificación	Apellidos	Nombres	S. Act.	Acciones	
015	1000765894	Arenas Quadros	Juan Sebastian	3	matricular	eliminar
019	1002658412	Bedoya Henao	Ana Julia	5	matricular	eliminar
026	254698	McArthur Oneill	Douglas	7	matricular	eliminar
035	1003874512	Atehortua Gil	María Eugenia	3	matricular	eliminar
036	1123564871	Lizcano Pérez	José Fernando	4	matricular	eliminar

[<<](#) [<](#) [1](#) [2](#) [3](#) [4](#) [>](#) [>>](#)

Observe que esta vista tiene todas las opciones para la gestión de estudiantes: agregar un **nuevo estudiante**, **consultar** haciendo clic en el Id. del estudiante, **modificar**, y **eliminar**. Además tiene una **barra de navegación** en la parte inferior.

Importante: Se deben desarrollar/programar todas las opciones del CRUD para esta vista.

Además, cada fila contiene el botón **matricular** que debe permitirle matricular los cursos para próximo semestre/nivel.

Cuando el usuario haga clic en **matricular**, se debe abrir una nueva vista de tipo formulario que permita matricular un nuevo curso para el estudiante seleccionado y guardar el registro en la tabla **cursos_matriculados**.

Ejemplo de vista para matricular un nuevo curso:

Proceso de matricula de cursos	
Estudiante:	Arenas Cuadros Juan Sebastian
Curso:	Cálculo Vectorial
Periodo:	2025-1
Estado:	Activo
Nota final:	0.0
guardar	

Tener en cuenta: Se matriculan cursos NO asignaturas porque cada curso identifica una asignatura pero también el profesor, el aula, el horario, etc. Ver el modelo entidad relación.

Importante: se deben cumplir las siguientes tres (3) condiciones para poder matricular un curso para el siguiente periodo.

- No puede matricular una asignatura que tenga un prerrequisito que no se haya cursado.
Por ejemplo: No se podrá matricular Cálculo Vectorial si no ha cursado Cálculo Integral

b. No se podrá matricular una asignatura que ya haya sido cursada (que aparezca en la tabla **asignaturas_cursadas**).

c. El número máximo de créditos que un estudiante puede cursar es **22**. Observe que la tabla **asignaturas** tiene un campo llamado **numero_creditos**.

En el listado de estudiantes, cuando el usuario haga clic en el Id. de un estudiante, se debe abrir una nueva vista para consultar los datos del estudiante y las asignaturas matriculadas para el siguiente período:

Consulta de estudiantes

Estudiante:

Arenas Cuadros Juan Sebastian

Semestre actual:

3

Cursos matriculados

Curso	Profesor	Aula	Horario	X
Cálculo Vectorial	Carlos Mário Álvarez	01-1504	V - 18:00	X
Ingeniería del Software 2	Gloria María Zuluaga	01-0506	W - 18:00	X
Lenguaje de Programación 2	Cesar Augusto Giraldo	02-0601	V - 09:00	X
Total créditos:				10

regresar

Importante: Desarrolle (opcionalmente) el resto del Front que crea necesario.

Muy importante: Las tablas de la base de datos deberían ser generadas desde el ORM de la tecnología del servidor que esté utilizando. Sólo como ejemplo, en Java/Spring debería ser generado con Spring Data-JPA. En todo caso, los datos de las demás tablas deben estar presentes en el momento de ejecutar la aplicación, me explico: se deben poblar los datos de las tablas **programas_academicos**, **planes_estudio**, **asignaturas**, algunos **estudiantes**, **asignaturas_cursadas** por esos estudiantes, **profesores**, **cursos**, algunos **cursos_matriculados**.

Los demás elementos que se evaluarán para esta suficiencia:

2. Utilizar tecnologías de servidor de Microsoft, de JavaEE o de Python.
3. 3. Hacer uso evidente de patrones de diseño como MVC, DAO, DTO, Inyección de Dependencias (DI) u otros de acuerdo a la necesidad.
4. Utilizar frameworks que permitan un desarrollo más profesional del producto esperado, como .net de Microsoft, Spring o JSF de Java o Django de Python y Bootstrap o Tailwind para la capa de presentación u otro similar.
5. Las bases de datos soportadas pueden ser MySQL, Posgresql o SQL Server

6. Utilizar librerías o frameworks de persistencia como JPA o Hibernate para Java o las propias de Microsoft o Python.
7. Permitir carga de archivos al servidor de archivos en formato pdf, jpg o png
8. Desarrollar un módulo de la aplicación para ofrecer servicios API REST, por ejemplo, para consumir el listado de **Asignaturas** o consultar un **Estudiante** o **Profesor** por su Id.
9. Implementar por lo menos dos reportes (exportación de información a pdf, word, excel, xls, json u otro), uno de ellos transaccional, el de la **Estudiantes** y sus **Cursos matriculados**.
10. Despliegue en un servidor de prueba externo en una nube de Google o de AWS o Heroku

Entregables.

1. El código fuente de la aplicación (manual del programador)
2. El diagrama de clases
3. Un prototipo funcional de la aplicación con todos los requerimientos anteriores.
 - CRUD para **Estudiantes y Cursos matriculados**, los demás son opcionales
 - Reportes en pdf, word, excel, xls, json u otro
 - Servicios API REST
 - Despliegue en servidor externo
4. Los scripts de la base de datos

Nota: Se solicitará opcionalmente, la sustentación del trabajo entregado.

Por último, agregar un servicio de login:

Ejemplo de solicitud de credenciales

Digitar credenciales

Ingresar

Puede ser almacenado en memoria o en base de datos, en este último caso, agregar la tablas necesarias.

Muchos éxitos